

Quadern del docent per sessions

Un full per sessió lectiva (SA1–SA9) amb la kata del dia, el mini-check individual i el repte ★ (quan toquen) i el checklist docent d'aquella sessió. Generat automàticament dels documents font — no els substitueix: si canvia algun d'ells cal tornar a executar `generar_fulls_imprimibles.py`.

SA1 · Sessió 1 — Què és un robot?

Checklist docent

- ☐ Activació: "*Quins robots tens a casa sense saber-ho?*"
- ☐ Model entrada→procés→sortida i anàlisi de 3 sistemes (Act. 1)
- ☐ Passar la **prova diagnòstica** (no qualifica → orienta el ritme)
- ☐ Tancament: presentar mètode de projecte (el quadern tècnic s'obre a la Sessió 2)
-  *Error a vigilar:* confondre entrada (sensor) amb sortida (actuador)

SA1 · Sessió 2 — Arquitectura i seguretat

Checklist docent

- ☐ micro:bit real a la mà; etiquetar l'esquema de la placa (Act. 2)
- ☐ **Signatura del full de seguretat** (recollir-lo)
- ☐ Tour de l'editor python.microbit.org (simulador + transferència del `.hex`)
- ☐ **Obrir el quadern tècnic** (primera entrada)



Mantra: la matriu de LED és
entrada

i sortida (mostra imatges i
llegeix llum) ·

Error: pensar que és només
decorativa

SA1 · Sessió 3 — El primer programa (PRIMM)

Checklist docent

- ☐ Mini-check individual (10', a l'inici — banc SA1)
- ☐ Projectar `hola_mon.py` **sense executar-lo** → alumnat prediu (Act. 4)
- ☐ Executar → Investigar → Modificar → **Crea** `emocions_botons`
- ☐ **Repte** ★ (Reptes_SA1.md, 25' fila pròpia) + 🧡 parella de lectura (5', dins)
- ☐ Mini-debat ètic (ODS) + presentar la fitxa-pòster
- ⚠️ *Error:* oblidar `from microbit import *` i no saber interpretar el `NameError`

Mini-check individual

Sense mirar `hola_mon.py`, **escriu de zero** un programa curt que: **(1)** mostri el teu nom lletra a lletra, **(2)** esperi mig segon, i **(3)** mostri fixa una imatge `Image.HAPPY` durant 2 segons.

★ Repte ★ — Targeta de benvinguda digital

Context. L'escola vol una petita "targeta" digital de presentació per al dia de portes obertes: cada alumne/a en fa una amb el seu nom.

Client: secretaria del centre · Lliurable: targeta digital de presentació · Món real: senyalització i identificació personal (targetes d'accés, etiquetes intel·ligents).

Què treballa. `display.scroll()`, `display.show()`, `Image`, `sleep()`.

Requisit mínim. - El display mostra el teu **nom** (`scroll`) i, després, una **imatge fixa** que et representi (`show` amb una `Image` predefinida). - Codi comentat, partint de `hola_mon.py`.

SA2 · Sessió 1 — Sortides digitals amb bucles

Checklist docent

- ☐ Encaixar el Micro:shield; muntar el LED extern a P1
- ☐ Modelatge de `led_parpelleig.py`: bucle + acumulador de comptador
-  *Error a vigilar:* inicialitzar el comptador dins del bucle (torna a zero cada volta)

Kata del dia

K01

Aquestes línies formen un programa vàlid però estan **desordenades**. Escriu-les en l'ordre correcte, amb la indentació que toqui, perquè: mentre es manté premut el botó A es mostri una cara contenta, i en cas contrari una cara adormida, per sempre. `sleep(100)` from microbit import * while True: if button_a.is_pressed(): display.show(Image.HAPPY) else: display.show(Image.ASLEEP)

SA2 · Sessió 2 — Sortides PWM i so

Checklist docent

- ☐ **Mini-check individual** a l'inici (10', banc SA2) — sortida digital, no PWM
- ☐ Modelatge de `pwm_led_rgb.py` (respiració + colors) i `musica_altaveu.py` (melodia + to)
-  *Error a vigilar:* confondre l'escala 0-1023 de `write_analog` amb 0-255

Kata del dia

K18

Completa els _____ perquè, en sacsejar la placa, el programa mostri un número de dau (1-6) a l'atzar. ``python from microbit import * import random
while _____: if accelerometer.was_gesture("shake"): numero = random.randint(1, _____) display.show(_____(numero)) sleep(1000) ``

Mini-check individual

Escriu **de memòria** un programa que faci parpellejar el LED del pin **P1**: mig segon encès, mig segon apagat, per sempre.

SA2 · Sessió 3 — Repte «semàfor o llum d'ambient» (producte)

Checklist docent

- ☐ Introduir el relé: seguretat (costat extern mai en contacte amb la placa)
- ☐ Acompanyar el muntatge i la programació individual del repte
- ☐ **Repte** ★ (25', fila pròpia — Reptes_SA2.md) + 🍷 parella de lectura (5', dins)
- ☐ **Mini-defensa oral (1', R4·DO), per MOSTREIG rotatiu:** 5-6 alumnes (registre rotatiu, vegeu 00_Guia_defensa_oral.md)

⚠️ *Retallada del pla de contingència:* si cal recuperar temps, aquest repte **fa de producte final** i la S4 s'allibera sencera

Kata del dia

K02

Ordena aquestes línies perquè el programa compti les sacsejades de la placa i mostri el comptador cada cop que en detecta una. `sleep(100) comptador = 0 from microbit import * while True: comptador = comptador + 1 display.show(str(comptador)) if accelerometer.was_gesture("shake"):`

★ Repte ★ — Llum de seguretat per a motxilla

Context. Una marca de motxilles vol un pilot LED d'emergència per a ciclistes/vianants: fix mentre tot va bé, parpelleig ràpid en "mode alerta".

Client: fabricant de motxilles i accessoris ciclistes · Lliurable: pilot LED de dos modes · Món real: llums de seguretat de bicicleta, senyalització d'obres.

Què treballa. `pin.write_digital()`, bucles `for/while`, acumuladors, `button_a/button_b`.

Requisit mínim. - Parteix de `led_parpelleig.py`: el LED parpelleja i es compta el nombre de cops (ja fet). - Afegeix un **segon mode**: mentre el **botó A** està premut, el LED parpelleja **més ràpid** ("mode alerta"); sense prémer'l, parpelleja al ritme normal. - Codi comentat.

SA2 · Sessió 4 — Fabricació i muntatge de la mascota

Checklist docent

- ☐ Repartir peces pretallades i personalitzades
 - ☐ Acompanyar el muntatge (dossier 00_Projecte_T1_Mascota.md)
 - ☐ Prova d'encesa: `led_parpelleig.py` (LED a P1) i `musica_altaveu.py` (so) funcionant; servo **muntat però no programat** (arriba a SA4)
 - ☐ Checklist de muntatge i **retorn ordenat del material** sobrant
-  *Mantra:* el relé **commuta** , no alimenta, el circuit extern

Kata del dia

K03

Ordena aquestes línies perquè una cara contenta parpellegi exactament **5 vegades** fent servir un `for`.

```
sleep(300) display.clear() from microbit import * VEGADES = 5 sleep(300) for i in range(VEGADES): display.show(Image.HAPPY)
```

SA3 · Sessió 1 — Entrades digitals i condicionals

Checklist docent

- ☐ Modelatge en directe al REPL: `is_pressed()/read_digital()`, *pull-up*, comptador de premudes
- ☐ Introduir el concepte d'**antirebot** (*debounce*) abans que aparegui com a "error"
-  *Error a vigilar:* llegir el polsador sense *pull-up* (valors erràtics)

Kata del dia

K04

Sense mirar apunts, completa aquest esquelet (màxim 4 línies noves) perquè el LED del pin **P1** estigui encès mentre es manté premut el botó A, i apagat en cas contrari. ``python from microbit import *
while True: # escriu aquí 3-4 línies sleep(50) ``

SA3 · Sessió 2 — Entrades analògiques: llum i temperatura

Checklist docent

- ☐ **Mini-check individual** a l'inici (10', banc SA3) — `if/else` sobre `read_light_level()`
- ☐ Modelatge de `nivell_llum.py` (barres, funció `mapa()`) i `termometre.py` (condicionals)
- ☐ Recordar els **pins ADC vàlids** (P0, P1, P2, P3, P4, P10) i que P3/P4/P10 no es poden llegir amb el display actiu (usar P0/P1/P2 a les pràctiques d'avui)
-  **Error a vigilar:** confondre l'escala 0-255 (sensors interns) amb la 0-1023 (pins ADC)

Kata del dia

K19

Completa els _____ perquè el programa compti cada vegada que es prem el botó A i mostri el comptador actualitzat. ``python from microbit import *

comptador = _____

while True: if button_a.was_pressed(): comptador = comptador _____ 1 display.
_____ (str(comptador)) sleep(100) ``

Mini-check individual

Tens `llum = display.read_light_level()` (0-255) al bucle principal. **(a)** Escriu l'`if/else` perquè el LED de P1 s'encengui quan `llum` sigui **més petit que 50** i s'apagui altrament. **(b)** Entre quins valors es mou `llum`?

SA3 · Sessió 3 — Repte «mascota reactiva» (producte — tanca T1)

Checklist docent

- ☐ Modelatge de `alarma_ultrasons.py`: `trigger/echo`, `machine.time_pulse_us`, seguretat 5 V
- ☐ Introduir el PIR: temps d'estabilització (30-60 s)
- ☐ Acompanyar el cablatge **EXACTE** de la mascota (P1, P2, P8, P12) i la programació individual
- ☐ **Repte** ★ de `Reptes_SA3.md` (nucli obligatori, en acabar la mascota, 25' fila pròpia): dins la S3 o com a deures abans de la S4
- ☐ **Mini-defensa breu (R4-DO), per MOSTREIG rotatiu**: 5-6 alumnes (reaccions + una decisió justificada; registre rotatiu, vegeu `00_Guia_defensa_oral.md`)
- ☐ Checklist de **tancament del Projecte T1**

⚠ *Mantra*: el cablatge de la mascota és **vinculant** : cap component nou fora de la taula del dossier

Kata del dia

K05

Ordena aquestes línies perquè el programa mostri una cara adormida quan hi ha poca llum (per sota del llindar) i esborri el display en cas contrari. `sleep(200)` `LLINDAR_FOSCOR = 50` `display.clear()` from `microbit import *` while True: `llum = display.read_light_level()` if `llum < LLINDAR_FOSCOR`: `display.show(Image.ASLEEP)` else:

★ Repte ★ — Llum automàtica d'estudi

Context. Una marca de làmpades d'escriptori vol un prototip que s'encengui sol quan la taula es queda fosca, sense que calgui prémer cap interruptor.

Client: fabricant de làmpades d'escriptori · Lliurable: llum automàtica de dos nivells · Món real: enllumenat automàtic d'exterior i passadissos.

Què treballa. `read_analog()` / `read_light_level()`, funció `mapa()`, condicionals, LED.

Requisit mínim. - Parteix de `nivell_llum.py`: lectura del sensor de llum (intern o extern) i barres al display (ja fet). - Afegeix un **LED extern** (pin lliure) que s'encengui quan la llum estigui **per sota** del teu `LLINDAR_FOSCOR`, i s'apagui altrament. - Codi comentat.

SA3 · Sessió 4 — Prova pràctica T1 (individual, sessió sencera)

Checklist docent

- ☐ Repartir l'enunciat de `Avaluació/Prova_practica_T1.md`
- ☐ Condicions de la prova: individual, sense apunts/ajuda, temps tancat
- ☐ Recollir les entregues i el material

SA4 · Sessió 1 — Funcions amb paràmetres i valor de retorn

Checklist docent

- ☐ Repassa un fragment ja conegut (`mapa()` de SA3, `respira()` de SA2) com a "funció que ja fèieu sense el nom"
- ☐ Modelatge de `funcions_moviments.py`: `graus_a_pwm()` (retorn) vs `mou_servo()` (sense retorn)
-  *Error a vigilar:* confondre una funció que **calcula** amb una que **fa** alguna cosa

Kata del dia

K06

Sense mirar apunts, completa aquest esquelet (màxim 4 línies noves) perquè es mostri una fletxa cap a la dreta (`Image.ARROW_E`) quan es faci el gest `'right'` amb la placa, i el display s'esborri en cas contrari. ``python

```
from microbit import *  
while True: # escriu aquí 3-4 línies sleep(100) ``
```

SA4 · Sessió 2 — Motoreductor amb funcions de moviment

Checklist docent

- ☐ Introduir els pins **definitius** M1/M2: no es tornaran a tocar en tot el curs
- ☐ Modelatge de `velocitat_pwm.py`: `avancar`/`retrocedir`/`girar`/`aturar`
- ☐ **Mini-check individual** al final de la sessió, amb el tancament (10', banc SA4) — funció amb paràmetre sense apunts

 *Error a vigilar:* motors alimentats només per USB (no giren); PWM als dos pins d'un motor alhora

Kata del dia

K20

Completa els _____ perquè el programa mostri una cara de sorpresa quan la temperatura superi els 28 graus, i esborri el display en cas contrari. ``python from microbit import *

LLINDAR_CALOR = _____

while True: temp = _____() if temp > LLINDAR_CALOR: display.show(Image.SURPRISED)
_____: display.clear() sleep(200) ``

Mini-check individual

Escriu una funció `avancar(velocitat)` que faci moure el motor esquerre (canal M1) a la velocitat rebuda com a paràmetre. **(b)** En una frase: per què el motoreductor **no** es pot alimentar només amb el corrent del port USB?

SA4 · Sessió 3 — Repte «control per botons» (producte)

Checklist docent

- ☐ PRIMM amb `control_per_botons.py` (predicció abans d'executar)
- ☐ Acompanyar la programació individual de la seqüència pròpia
- ☐ **Repte** ★ de `Reptes_SA4.md` (nucli obligatori, diferent de «control per botons», 25' fila pròpia): dins la S3 o com a deures abans de la S4
- ☐ **Mini-defensa breu (R4-DO), per MOSTREIG rotatiu:** 5-6 alumnes (seqüència + una decisió justificada; registre rotatiu, vegeu `00_Guia_defensa_oral.md`)

⚠ **Mantra:** el botó B **sempre** atura, es processa on es processa (anticipa l'STOP de la SA6)

Kata del dia

K07

Ordena aquestes línies perquè defineixin una funció `saluda(vegades)` que repeteix un gest content tantes vegades com li digui el paràmetre, i que després es cridi 3 vegades. `saluda(3) display.clear()`
`sleep(300)` `def saluda(vegades): sleep(300) for i in range(vegades):`
`display.show(Image.HAPPY)`

★ Repte ★ — Salutació programable per a un aparador

Context. Una botiga vol una figura d'aparador que saludi els vianants de manera diferent segons el moment del dia (matí animat, tarda tranquil·la), sense que calgui reprogramar-la cada cop.

Client: comerç de barri · Lliurable: figura d'aparador amb salutacions configurables · Món real: cartells i figures animatròniques d'aparador, robots de recepció.

Què treballa. Funcions amb paràmetres, servo amb PWM, `mou_servo()` / `saluda()` de `funcions_moviments.py`.

Requisit mínim. - Parteix de `funcions_moviments.py`: `saluda(vegades)` i `escombra(angle_maxim)` (ja fet). - Escriu una funció nova `salutacio(estil, vegades)` amb **dos** paràmetres: `estil` ('curt' o 'llarg') decideix si crida `saluda()` o `escombra()`. - Codi comentat.

SA4 · Sessió 4 — Muntatge del vehicle T2 (fabricació)

Checklist docent

- ☐ Acompanyar el muntatge pas a pas del dossier del vehicle
 - ☐ Supervisar la **prova de fum** (motors giren en el sentit esperat) abans de tancar el xassís
 - ☐ Checklist de muntatge; recordar portar el vehicle a la SA5
-  *Mantra:* els pins de motor **no canvien** respecte a la S2-S3; el cablatge és mecànic, no de programació

SA5 · Sessió 1 — Xat per ràdio

Checklist docent

- ☐ Repartir/repassar la taula de grups de ràdio
- ☐ Modelatge de `radio_missatges.py`: `radio.on()`, `radio.config(group=...)`, `send()/receive()` dins del bucle
- ☐ Deixar clar que l'aparellament és **puntual** (banc de proves), no producte compartit
-  *Error a vigilar:* grups diferents entre parelles (no reben res) o coincidents amb una altra parella (interferències)

Kata del dia

K08

Ordena aquestes línies perquè una funció converteixi un angle (0-180°) al valor PWM que espera un servomotor, i mostri el resultat per a un angle de 90°. `valor = graus_a_pwm(90)` `def graus_a_pwm(angle):`
`display.scroll(str(valor)) return 26 + (angle * (128 - 26)) // 180`

SA5 · Sessió 2 — Dissenyar un protocol de comandes

Checklist docent

- ☐ Introduir la idea de **protocol** (prefix + ordre curta) abans de mostrar `comandament.py`
 - ☐ Modelatge de la connexió recepció → funcions de moviment de la SA4
 - ☐ **Mini-check individual** al final de la sessió, amb el tancament (10', banc SA5) — enviar/rebre un missatge i actuar-hi
-  *Error a vigilar:* `PREFIX` que no coincideix exactament entre comandament i receptor

Kata del dia

K21

Completa els _____ perquè la funció mostri una fletxa cap a l'esquerra o cap a la dreta segons el paràmetre rebut, i crida-la amb `'dreta'`.
``python def mostra_fletxa(costat): if costat == 'esquerra':
display.show(_____) elif costat == _____:
display.show(Image.ARROW_E)
_____('dreta') ``

Mini-check individual

Escriu de zero el bucle receptor: configura la ràdio en el canal (group) **10**, activa-la, i a cada volta comprova si ha arribat el missatge exacte `'F'`; si és així, crida `avancar(400)`.

SA5 · Sessió 3 — Repte «control remot bàsic» (producte)

Checklist docent

- ☐ PRIMM amb `receptor_vehicle.py` (predicció abans d'executar)
 - ☐ Acompanyar el tancament individual del repte, aparellat puntualment amb un company o el docent com a emissor
 - ☐ **Repte** ★ de `Reptes_SA5.md` (nucli obligatori, 25' fila pròpia)
 - ☐ **Mini-defensa breu (R4-DO), per MOSTREIG rotatiu:** 5-6 alumnes (protocol + una decisió justificada; registre rotatiu, vegeu `00_Guia_defensa_oral.md`)
- ⚠ **Mantra:** les funcions de moviment **no canvien** respecte a la SA4; només canvia l'entrada (ràdio en lloc de botons)

Kata del dia

K09

Ordena aquestes línies perquè una funció canviï una variable `emocio` definida fora d'ella, i mostri el nou valor.

```
canvia_emocio(1) emocio = 0 global emocio def canvia_emocio(nova): display.show(str(emocio))
emocio = nova
```

★ Repte ★ — Xat de classe amb identificació

Context. Un club de robòtica vol un petit "walkie-talkie" digital: cada membre s'identifica amb un nom curt en cada missatge, perquè es pugui saber sempre qui ha enviat què.

Client: club de robòtica escolar · Lliurable: xat amb identificació de remitent i historial · Món real: aplicacions de missatgeria, ràdios d'equip amb identificador de canal.

Què treballa. `radio.on()/config()/send()/receive()`, llistes, `radio_missatges.py`.

Requisit mínim. - Parteix de `radio_missatges.py`: enviament amb remitent i historial en una llista (ja fet). - Afegeix una funció `mostra_historial()` que faci `display.scroll()` de **tots** els missatges de `historic`, un darrere l'altre. - Codi comentat.

SA6 · Sessió 1 — Llaç obert i llaç tancat: la màquina d'estats

Checklist docent

- ☐ Diferenciar llaç obert/tancat amb exemples propis del vehicle (SA4-SA5) abans del codi
- ☐ Acompanyar el disseny del **diagrama RUN/STOP/ALERTA** al paper abans d'escriure res
- ☐ Modelatge de `maquina_estats_semafor.py` i `termostat_histeresi.py` (histeresi: per què dos llindars)
-  *Error a vigilar:* un sol llindar al termòstat (provoca "clic-clic" a l'observació en directe)

Kata del dia

K10

Ordena aquestes línies perquè el programa envii "Hola" en prémer el botó A, i mostri qualsevol missatge que arribi.

```
radio.on()
import radio
sleep(20)
radio.config(group=5)
if button_a.was_pressed():
while True:
    radio.send("Hola")
    missatge = radio.receive()
    if missatge is not None:
        display.scroll(missatge)
```

SA6 · Sessió 2 — Aturada d'emergència prioritària

Checklist docent

- ☐ **Mini-check individual** a l'inici (10', banc SA6) — histèresi, sense apunts
 - ☐ Modelatge de `actualitza_estat()`: STOP com a únic lloc que atura motors, mostra LED i display
 - ☐ Comprovar que el polsador es mira **abans** que qualsevol altra entrada a cada volta del bucle
 - ☐ Introducció breu de `registre_dades.py` (mòdul `log`, lectura per USB)
-  *Error a vigilar:* comprovar el polsador només "de tant en tant" en lloc de cada iteració

Mini-check individual

Un termòstat activa el relé quan `temp < 25` i el desactiva quan `temp >= 25`. A la pràctica, el relé fa **clíc-clíc sense parar** al voltant de 25 °C. **(a)** Per què passa? **(b)** Reescriu les condicions perquè no passi (pista: dos llindars).

SA6 · Sessió 3 — Repte «vehicle amb aturada d'emergència» (producte, tanca el Projecte T2)

Checklist docent

- ☐ PRIMM amb `vehicle_seguretat.py` (predicció abans d'executar): què fa "X" si arriba en ple moviment
- ☐ Acompanyar el tancament individual: protocol complet F/B/L/R/S/X + STOP prioritari
- ☐ **Repte** ★ de `Reptes_SA6.md` (nucli obligatori, 25' fila pròpia, termòstat de dues zones)
- ☐ **Mini-defensa breu (R4-DO), per MOSTREIG rotatiu:** 5-6 alumnes (FSM + una decisió justificada; registre rotatiu, vegeu `00_Guia_defensa_oral.md`)
- ☐ Ampliació opcional: sensor de temperatura/DHT11 com a tercer estat ALERTA (**no nucli**)
- ⚠️ **Mantra:** les funcions de moviment **no** respecte a SA4/SA5; només s'hi afegeix la màquina i el protocol **canvien** d'estats amb STOP prioritari

Kata del dia

K11

Ordena aquestes línies perquè una funció guardi missatges en una llista, però descarti sempre el més antic quan ja n'hi hagi més de MAX. `desa("A")` `historic = []` `historic.append(missatge)` `MAX = 3` `def` `desa(missatge):` `historic.pop(0)` `if len(historic) > MAX:` `desa("B")`

★ Repte ★ — Termòstat de dues zones

Context. Un petit hivernacle escolar té dues zones (semillers i planter) amb necessitats de temperatura diferents. El responsable de l'hort de l'institut vol un sistema que avisi de quina zona necessita escalfor sense que el relé "cliqui" contínuament.

Client: hort escolar de l'institut · Lliurable: termòstat amb dues zones simulades i histèresi pròpia per zona · Món real: control climàtic d'hivernacles, incubadores.

Què treballa. Histèresi (dos llindars), variables d'estat múltiples, `termostat_histeresi.py`.

Requisit mínim. - Parteix de `termostat_histeresi.py`: manté la histèresi de dos llindars (ja feta). - Simula una **segona zona** amb una segona parella de llindars (per exemple, `LLINDAR_BAIX_2/LLINDAR_ALT_2`) i un segon "actuador" (un altre pin digital o el display mostrant quina zona està activa). - Codi comentat.

SA6 · Sessió 4 — Prova pràctica T2 (individual, sessió sencera)

Checklist docent

- ☐ Repartir l'enunciat i el material individual necessari
- ☐ Supervisar en silenci; només aclarir dubtes d'enunciat, no de solució
- ☐ Recollir el material i les entregues al tancament

SA7 · Sessió 0 — Muntatge del rover

Checklist docent

- ☐ Repartir el joc de les dues peces noves (ja impreses pel docent) a cada alumne/a
- ☐ Acompanyar el muntatge: suport HC-SR04 al davant, suport seguidor de línia sota el xassís
- ☐ Passar la **checklist de muntatge (R2, formativa)**: GND comú, sensors ben orientats, motors intactes
-  *Pla B:* peça endarrerida → fixar temporalment amb cinta/brides i seguir programant

SA7 · Sessió 1 — Cinemàtica diferencial

Checklist docent

- ☐ Revisar el rover muntat; recordar que `avancar/retrocedir/girar/aturar` són les mateixes de la SA4, sense pins nous
- ☐ Modelatge de `calibratge_motors.py`: per què cal compensar M1/M2
- ☐ Acompanyar el calibratge individual i les primeres proves de trajectòria (quadrat)
-  *Error a vigilar:* atribuir la desviació del rover a "un error de codi" en lloc del desequilibri real dels motors

Kata del dia

K12

Ordena aquestes línies perquè una petita màquina d'estats alterni "encès"/"apagat" cada segon, fent servir un diccionari de transicions. `sleep(1000)` `estat = APAGAT` `canvia()` `ENCES`, `APAGAT = range(2)` `global` `estat` `TRANSICIONS = {ENCES: APAGAT, APAGAT: ENCES}` `def canvia():` `while True:` `estat =` `TRANSICIONS[estat]` `display.show(str(estat))`

SA7 · Sessió 2 — Seguidor de línia

Checklist docent

- ☐ **Mini-check individual** a l'inici (10', banc SA7) — cicle llegir→decidir→actuar
- ☐ Modelatge de `segueix_linia.py`: `read_analog()` i llindar de detecció calibrat sobre el circuit real
- ☐ Acompanyar la calibració del llindar taula per taula (la llum de cada punt de l'aula varia)

 *Error a vigilar:* assumir un llindar universal vàlid per a tota la classe

Mini-check individual

El teu rover ja avança recte gràcies al calibratge de M1/M2 (Sessió 1). Imagina un sensor analògic qualsevol muntat al davant, llegit amb `SENSOR.read_analog()` (0-1023), amb un llindar ja calibrat `LLINDAR = 500`. Les funcions de moviment (`avancar()`, `aturar()`) ja estan fetes. Escriu el **bucle complet**: si la lectura és per sota del llindar, avança (`avancar(400)`); si és per sobre, atura't (`aturar()`).

SA7 · Sessió 3 — Evita-obstacles (repte «tria un comportament autònom»)

Checklist docent

- ☐ PRIMM amb `evita_obstacles.py` (predicció abans d'executar)
- ☐ Modelatge de `mesura_distancia()`: mateix patró que `alarma_ultrasons.py` (SA3), pins nous (trigger P1, echo P2)
- ☐ Acompanyar el repte «tria un comportament autònom» (línia i/o obstacles, segons material de taula)
-  *Mantra:* el mètode del sensor **no** canvia respecte a la SA3; només canvien els pins

Kata del dia

K13

Ordena aquestes línies perquè el programa recorri una llista de distàncies mesurades i mostri un avís cada cop que una és inferior a 10 cm. `sleep(300)` `distancies = [12, 8, 20, 5]` `display.clear()` `for d in distancies: display.show("!") if d < 10: else:`

SA7 · Sessió 4 — Missions del rover (producte)

Checklist docent

- ☐ Modelatge de `rover_missions.py`: selecció de missions amb botons + polsador STOP prioritari (mateix patró que `vehicle_seguretat.py`, SA6)
- ☐ Acompanyar la integració individual amb millores (velocitat variable, marge de seguretat)
- ☐ **Repte** ★ de `Reptes_SA7.md` (nucli obligatori, 25' fila pròpia, carret de magatzem amb velocitat variable)
- ☐ **Mini-defensa breu (R4-DO), per MOSTREIG rotatiu**: 5-6 alumnes (una decisió justificada; registre rotatiu, vegeu `00_Guia_defensa_oral.md`)

⚠ *Error a vigilar:* el polsador STOP no comprovat dins del bucle intern de cada missió

Kata del dia

K14

Ordena aquestes línies perquè el programa intenti convertir un missatge a número i, si no ho aconsegueix, mostri una "E" d'error en lloc d'aturar-se. `display.show("E")` `valor = int(missatge)` `missatge = "12"`
`try: display.show(str(valor))` `except ValueError:`

★ Repte ★ — Carret de magatzem amb velocitat variable

Context. Un petit magatzem escolar (el material de robòtica del centre) vol un carret que segueixi un carril pintat entre les prestatgeries, més ràpid als trams rectes i més lent a les corbes, perquè no caigui res del carret.

Client: coordinació de material del centre · Lliurable: seguidor de línia amb velocitat variable segons la qualitat de la lectura · Món real: carrets AGV de magatzem que segueixen carrils pintats o cintes magnètiques.

Què treballa. Llindar calibrat, correcció de rumb, `segueix_linia.py`.

Requisit mínim. - Parteix de `segueix_linia.py`: manté la lectura amb `read_analog()` i el llindar calibrat (ja fet). - Afegeix una **segona velocitat**, més baixa, que s'utilitzi durant els girs de correcció (quan es perd la línia), i la velocitat normal quan la segueix recte. - Codi comentat.

SA8 · Sessió 1 — Sensors avançats: llegir el Kit 3

Checklist docent

- ☐ Presentar els 4 sensors del Kit 3 i explicar per què el nucli programa només IMU+DHT11 (I2C i time-of-flight, ja coneguts)
- ☐ Modelatge de `comportaments.py`: repàs de la FSM SEGUIR/ESQUIVAR/RECUPERAR
- ☐ Acompanyar el muntatge del DHT11 (P8) i l'IMU (I2C, P19/P20) i el disseny del format de missatge
-  *Error a vigilar:* confondre "estat de la FSM" (decisió pròpia) amb "valor d'un sensor" (dada externa)

Kata del dia

K15

Ordena aquestes línies perquè el programa registri la temperatura i la llum cada 2 segons amb el mòdul `log` de la `micro:bit`. `sleep(2000)` `import log` `log.set_labels('temp', 'llum')`
`log.add(temp=temperature(), llum=display.read_light_level()) while True:`

SA8 · Sessió 2 — Telemetria per ràdio: enviar i registrar

Checklist docent

- ☐ Modelatge de `telemetria_radio.py`: FSM + sensors + `radio.send()` amb prefix `"TEL:"`
- ☐ Modelatge d'`estacio_base.py`: cada alumne l'escriu, encara que s'executi temporalment en una altra placa
- ☐ **Mini-check individual** a l'inici (10', banc SA8) — enviar un valor de sensor per ràdio
- ☐ Acompanyar el muntatge i les proves per parelles (torns de placa d'estació base)
- ⚠ *Error a vigilar:* `group` o `PREFIX` diferents entre les dues plaques d'una mateixa parella

Mini-check individual

Escriu de zero un emissor de telemetria: configura la ràdio en el canal **10**, activa-la, i cada **2 segons** envia la temperatura llegida amb `temperature()`, en forma de text que comenci per `'T:'`.

SA8 · Sessió 3 — IA aplicada al control i producte

Checklist docent

- ☐ Demostració/pràctica guiada de classificació de patrons (Teachable Machine), amb pla B sense internet
- ☐ Bloc «Ètica de dades i IA» (RGPD, biaix, consentiment) aplicat a la telemetria del rover
- ☐ Acompanyar el tancament del producte (mínim dos sensors, ràdio, registre)
- ☐ **Repte** ★ de Reptes_SA8.md (nucli obligatori, 25' fila pròpia, estació meteorològica amb alertes)
- ☐ **Mini-defensa breu (R4-DO), per MOSTREIG rotatiu:** 5-6 alumnes (una decisió justificada; registre rotatiu, vegeu 00_Guia_defensa_oral.md)

 *Error a vigilar:* que la reflexió d'IA es quedi en generalitats sense connectar-la amb `mpu_orientacio()`

Kata del dia

K16

Sense mirar apunts, escriu (4-6 línies) una funció `girar(costat, velocitat=300)` que faci girar el motor M1 (pins P13/P14) a la velocitat rebuda si `costat` val `'esquerra'`, i el motor M2 (pins P15/P16) si val `'dreta'`. Crida-la un cop passant els dos arguments, i un altre cop **sense** indicar la velocitat (per comprovar que fa servir el valor per defecte).

★ Repte ★ — Estació meteorològica escolar amb alertes

Context. La coordinació de manteniment d'un institut vol un petit sensor mòbil que avisi quan la temperatura d'una aula o d'un magatzem surt d'un rang segur, sense haver de mirar constantment cap pantalla.

Client: coordinació de manteniment del centre · Lliurable: telemetria amb alerta de temperatura per ràdio · Món real: sensors IoT de sales de servidors o magatzems que avisen quan la temperatura surt de rang.

Què treballa. Protocol de telemetria amb prefix, DHT11, llindar de temperatura, `telemetria_radio.py`.

Requisit mínim. - Parteix de `telemetria_radio.py`: manté la lectura del DHT11 i de l'IMU MPU6050 (ja fetes). - Afegeix un **llindar de temperatura** (`LLINDAR_TEMP_ALTA`) i un nou camp al missatge de telemetria (`"AL:1"` si se supera, `"AL:0"` si no). - Codi comentat.

Checklist docent

- ☐ Presentar el banc de reptes i el maquinari de reserva
- ☐ Acompanyar la tria individual del repte + requisits mínims + esbós
- ☐ Comprovar que ningú s'ha quedat sense repte viable amb el seu maquinari

 *Error a vigilar:* repte massa ambiciós per al temps disponible

Kata del dia

K17

Completa els _____ perquè la funció classifiqui una distància com a "PROP" (menys de 15 cm) o "LLUNY" (la resta), i mostri el resultat de classificar 10. `python def classifica(distancia): if distancia < 15: return "PROP" else: return _____`
`resultat = _____(10) display.scroll(str(resultat))`

SA9 · Sessió 2 — Prototipar

Checklist docent

- ☐ Mini-check individual (10', a l'inici — banc SA9)
- ☐ Modelatge de `plantilla_projecte.py` (percep/decideix/actua)
- ☐ Ronda de muntatge del component nou de cada repte (atenció especial: relé+bomba, PIR, NeoPixel)
- ☐ Acompanyar la primera integració de codi (prototip mínim viable)

 *Error a vigilar:* bomba connectada directament al Micro:shield, sense relé

Mini-check individual

Escriu una funció `comprova_i_envia()` que **integri** el que ja saps: llegeix un sensor (per exemple `accelerometer.get_x()`), i si el valor supera un llindar que tu decideixis, envia un missatge per ràdio amb `radio.send()`.

SA9 · Sessió 3 — Provar i millorar

Checklist docent

- ☐ Acompanyar proves sistemàtiques (DEPURA) i una 1a iteració de millora
- ☐ Reservar 20-30' finals per a **defenses esglaonades** de qui ja estigui llest
- ☐ Avançar el dossier tècnic (§1-§3)

 *Error a vigilar:* només es prova el "cas feliç", sense cap prova de límit

Kata del dia

K22

Completa els _____ perquè el programa intenti convertir a número el missatge rebut per ràdio i, si no ho aconsegueix, mostri "E" en lloc d'aturar-se. `python missatge = radio.receive() if missatge is not None: _____: valor = int(missatge) display.scroll(str(valor)) except _____: display.show("E")`

K23

Completa els _____ perquè la funció retorni el valor més alt d'una llista de temperatures. `python def maxim(llista): valor_maxim = llista[0] for valor in _____: if valor > _____: valor_maxim = valor return _____`
`temperatures = [21, 25, 19, 28, 22] display.scroll(str(maxim(_____)))`

K24

Completa els _____ perquè la funció compti quantes distàncies d'una llista són inferiors a 10. `python def compta_perill(distancies): comptador = _____ for d in _____: if d < _____: comptador = comptador + 1 return comptador`
`mesures = [12, 8, 5, 20, 9] display.scroll(str(compta_perill(_____)))`

SA9 · Sessió 4 — Comunicar

Checklist docent

- ☐ Acompanyar el tancament del dossier tècnic (§4-§9)
- ☐ Dirigir les **defenses orals individuals** (5' + preguntes), moderant el torn de manera equilibrada
- ☐ Recollir dossiers i repartir el repàs exprés (deures per a la T3)
-  *Error a vigilar:* dossier incomplet arribat a la defensa per manca de control en sessions anteriors

Kata del dia

K25

Completa els _____ perquè el programa envii una ordre d'aturada amb prefix "CMD:" en prémer A, i la reconegui correctament en rebre-la. ```python PREFIX = "CMD:"
if button_a.was_pressed(): radio.send(_____ + "S")
missatge = radio.receive() if missatge is not None and missatge._____(PREFIX): ordre =
missatge[_____]: if ordre == "S": aturar() ```

K26

Completa els _____ perquè la funció canviï d'estat segons el diccionari de transicions de tres estats. ```python SEGUIR, ESQUIVAR, RECUPERAR = range(3) estat = SEGUIR
TRANSICIONS = {SEGUIR: _____, ESQUIVAR: RECUPERAR, RECUPERAR:
_____}
def seguent_estat(): global estat estat = TRANSICIONS[_____] return estat
display.show(str(seguent_estat())) ```

K27

Completa els _____ perquè la funció retorni "FRED" si la temperatura és inferior a 18, "CALOR" si és superior a 26, i "NORMAL" en la resta de casos. ```python def zona(temp): if temp
_____ 18: return "FRED" elif temp _____ 26: return
_____ else: return "NORMAL"
display.scroll(zona(22)) ```

SA9 · Sessió 5 — Prova pràctica T3 (NO és sessió de projecte)

Checklist docent

- ☐ Organitzar estacions rotatives (taula + pista), torns de 8-10'
- ☐ Recordar: no reavalua el projecte; és destreses individuals de SA7-SA8